

ATOMQUEST

Goal Setting & Performance Management Portal

Hackathon Submission Document

Submitted by: **Sudarshan Khot**

1. Submission Links

Type	URL
Live Application	https://atom-quest-topaz.vercel.app/
Source Code	https://github.com/sudarshan-khot/AtomQuest

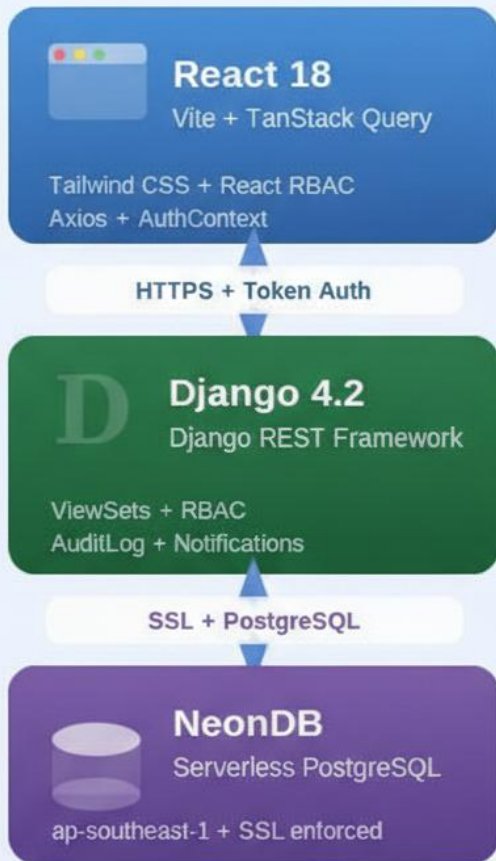
2. Technology Stack

Layer	Technology
Frontend	React 18 · Vite · TanStack Query v5 · React Router v6 · Tailwind CSS v3
Backend	Django 4.2 · Django REST Framework · Token Authentication
Database (Dev)	SQLite — local file, zero configuration
Database (Prod)	NeonDB — Serverless PostgreSQL · ap-southeast-1 · SSL required
Hosting	Vercel — Monorepo experimentalServices (static CDN + serverless Python)
Task Queue	Celery + Redis — configured, not active (cost-optimised for hackathon)
API Style	REST · DRF ViewSets · Token in Authorization header
State Management	TanStack Query (server state) · React Context (auth) · useState (UI)

3. Architecture Diagram

The diagram below illustrates the three-tier serverless architecture of AtomQuest. The left column shows each layer; the right panel highlights the key features. Every arrow represents an HTTPS call secured by DRF Token Authentication.

AtomQuest System Architecture



Key Features

Goal Lifecycle

Draft → Submit → Approve → Lock

Role-Based Access Control

Admin · Manager · Employee · Viewer

Quarterly Check-ins

Progress tracking with weighted scores

Shared Goals (KPI Distribution)

Push goals to entire departments

Audit Trail

Immutable log of every action

Reports & Analytics

Achievement scores · Charts · CSV export

4. Future Scaling Roadmap

The following enhancements were intentionally deferred to keep infrastructure cost at zero during the hackathon. Each is already partially scaffolded in the codebase and can be activated without architectural rewrites.

Enhancement	Implementation Note
Celery + Redis Task Queue	Currently configured but not active. Activating enables async email notifications, scheduled check-in reminders, and background report generation — no code changes required.

Enhancement	Implementation Note
Email / SMS Notifications	Notification model and dispatcher already in place. Plugging in SendGrid or AWS SES makes delivery external; keeps notification UX unchanged.
NeonDB Read Replicas	NeonDB supports read replicas with a connection-string swap. Reports and audit-trail queries can be routed to replicas, eliminating read load on the primary.
NeonDB Branching for Staging	Git-style database branches allow a full staging environment with production data snapshot — zero infrastructure cost.
CI/CD Pipeline	GitHub Actions → Vercel deploy hook; run Django test suite and Playwright E2E on each PR before production deploy.
Multi-tenancy / Org Isolation	Current architecture is single-tenant. Adding an Organisation model with FK on all entities and row-level-security policies enables SaaS multi-tenancy.
SSO / OAuth2 Login	DRF supports social-auth backends. Integrating Okta, Azure AD, or Google SSO removes password management and enables enterprise adoption.
Analytics Dashboard	Recharts is already imported in the frontend. Extending the <code>/api/reports/</code> endpoint to return aggregated time-series data enables richer trend charts with minimal backend work.